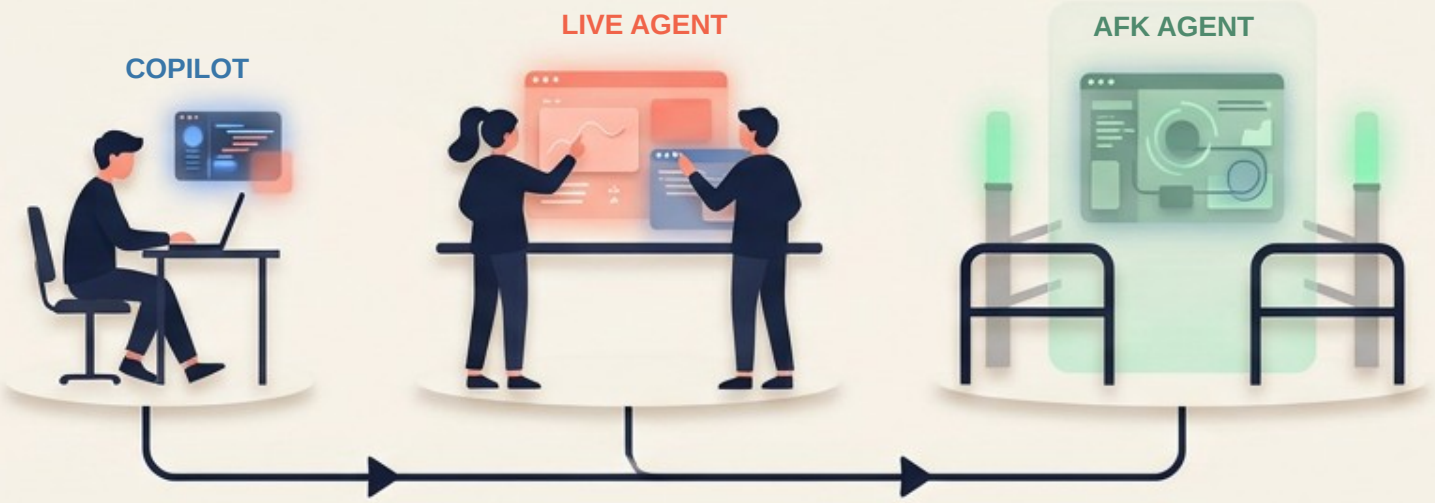
AI-Assisted Software Engineering

From coding assistance to reliable autonomous workflows



If AI writes code, why study CS?

AI can generate code.
You need to judge the result.

System design

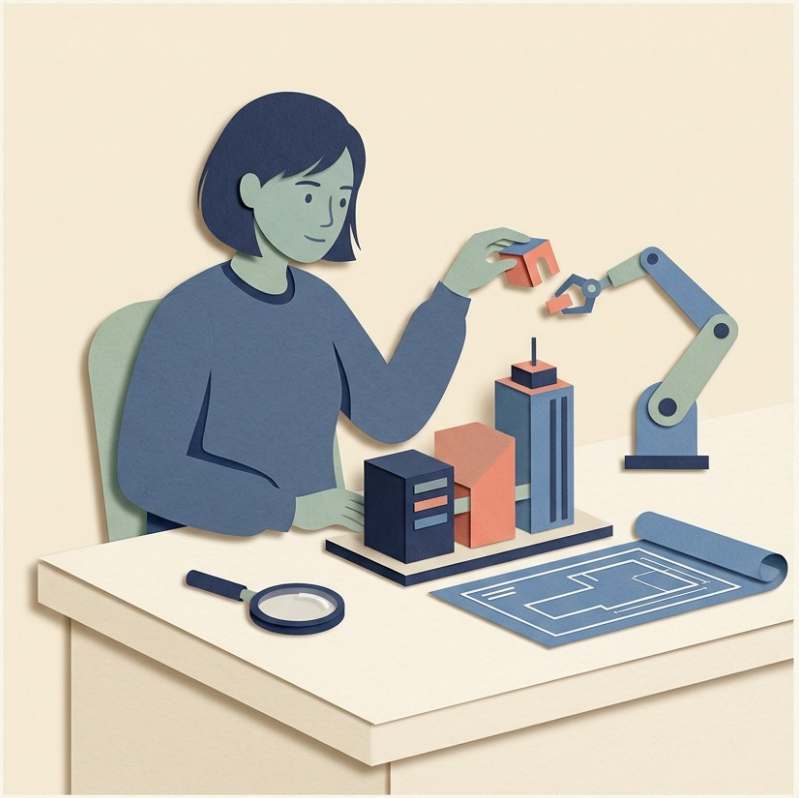
Choose boundaries and understand trade-offs.

Correctness

Recognize broken assumptions and test behavior.

Ownership

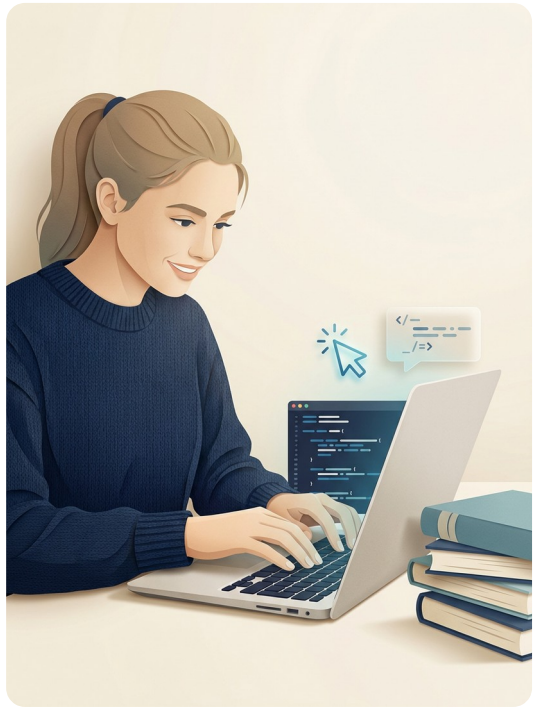
Explain, change and maintain what you ship.



One prompt. Many hidden decisions



Three levels of AI coding



COPILOT

You write the code.
AI suggests the next change.



LIVE AGENT

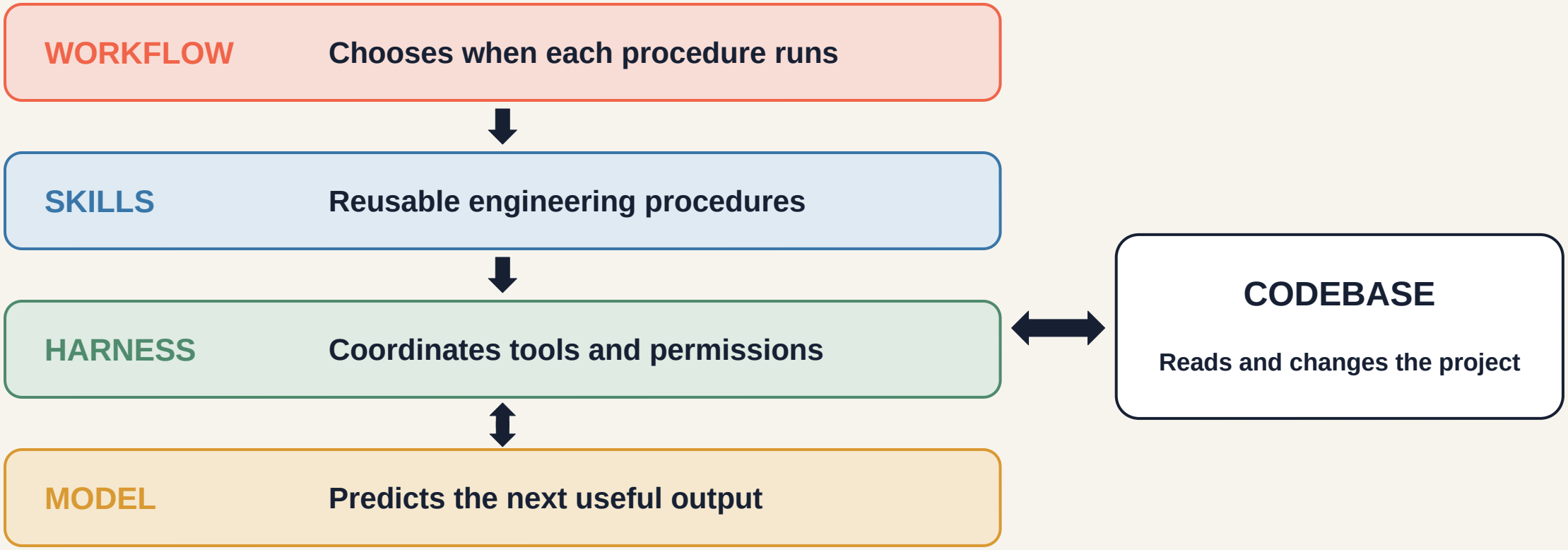
The agent edits and runs tools.
You stay present and steer.



AFK AGENT

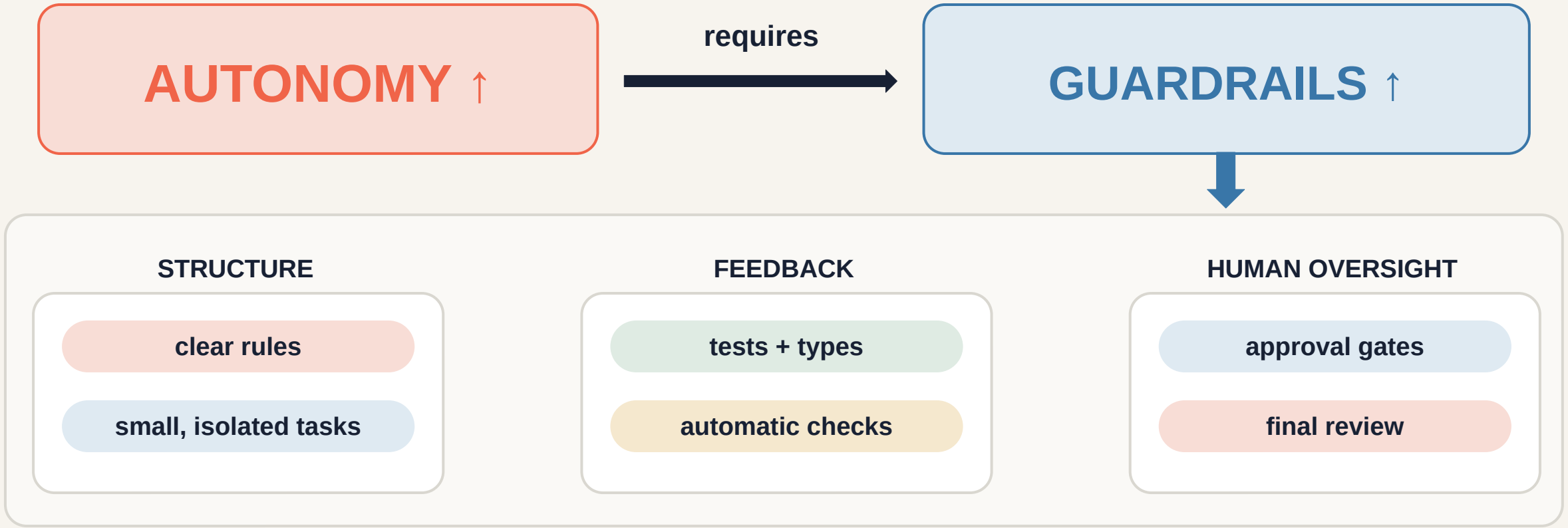
The agent works while you are away.
You review its evidence later.

What is actually doing the work?



The model predicts. The surrounding system turns predictions into work.

The autonomy equation



A better model helps. It does not remove the guardrail system.

Skills for real engineers

Not one giant prompt

Small, composable engineering skills

adaptable

versioned

human-controlled

A skill is a reusable operating procedure, not extra intelligence.

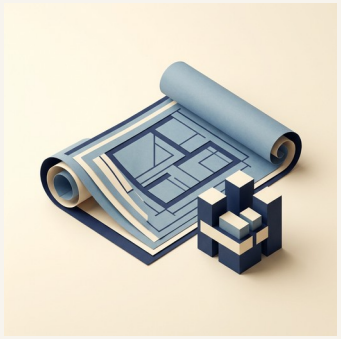


The entire journey



GRILL

decide



SPEC

preserve



TICKETS

slice



BUILD

prove



REVIEW

inspect



RETRO

improve

Step 1: Grill the idea

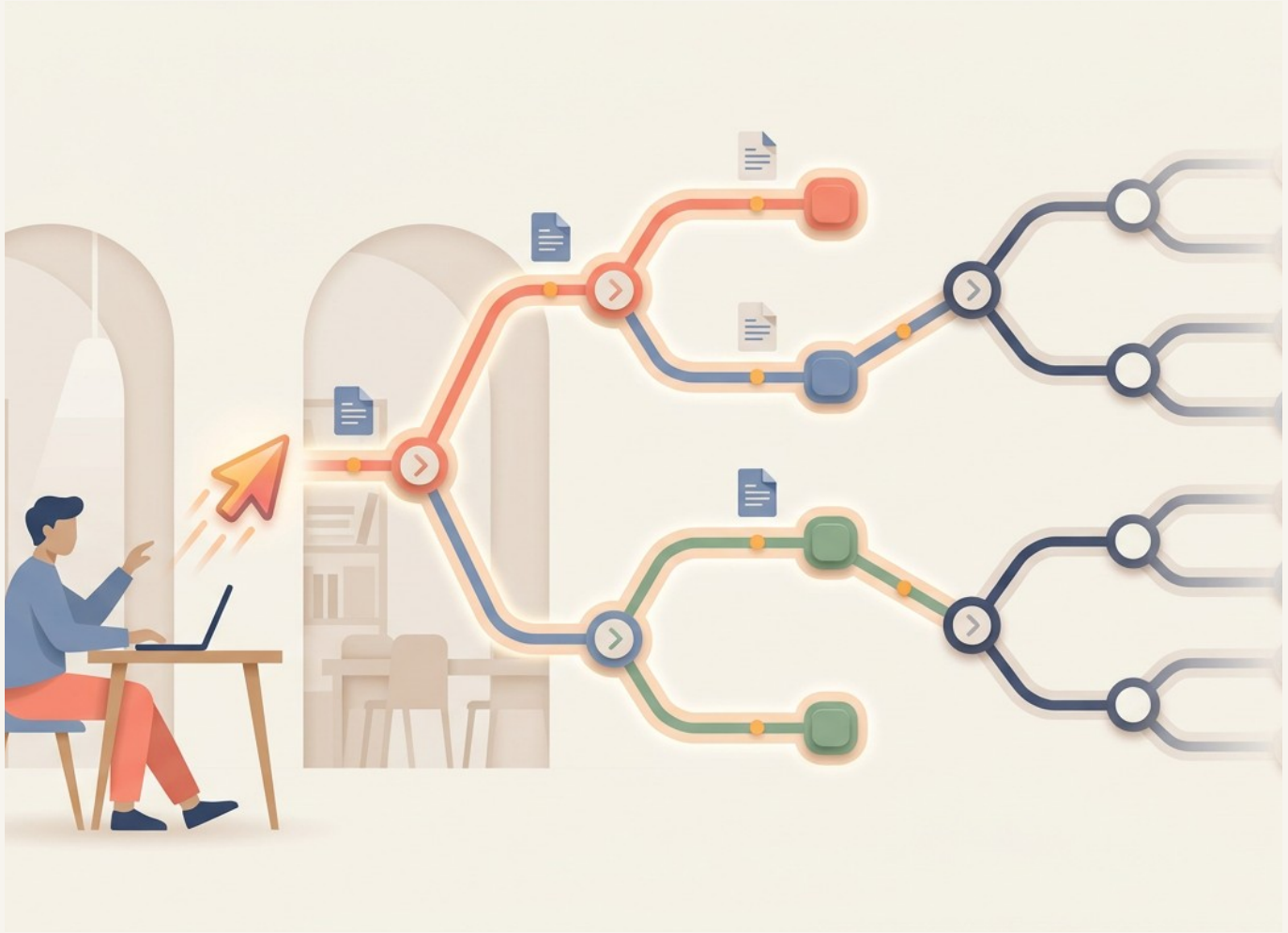
Facts

agent investigates

Decisions

human answers

Ask only what can be decided now



Shared words reduce repeated confusion

BEFORE

“A loan record that has not been returned yet”

AFTER

ACTIVE LOAN

GLOSSARY.md

ADRs for durable decisions

Name the domain once. Reuse the meaning everywhere.

Step 2: Preserve the destination

A spec remembers what the conversation decided

- USER BEHAVIOR
- TECHNICAL DECISIONS
- TEST SEAMS
- OUT OF SCOPE

Skip it when the entire change fits one live session

Blueprints before bricks: /to-spec



“Build borrowing”

Leaves success and failure undefined

A contract for one behavior

INPUT

An eligible member and an available book

SUCCESS

Create an active loan and mark the book unavailable

FAILURE

Reject an unavailable book without changing state

EVIDENCE

Tests check both outcomes through LibraryService

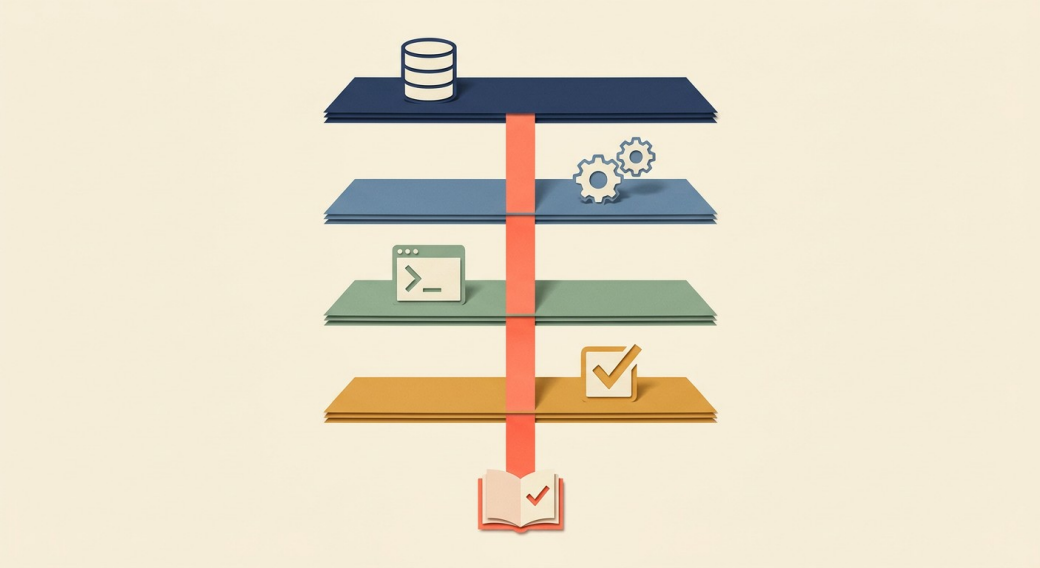
Step 3: Fire tracer bullets

HORIZONTAL LAYERS



Four polished parts
that do not work together

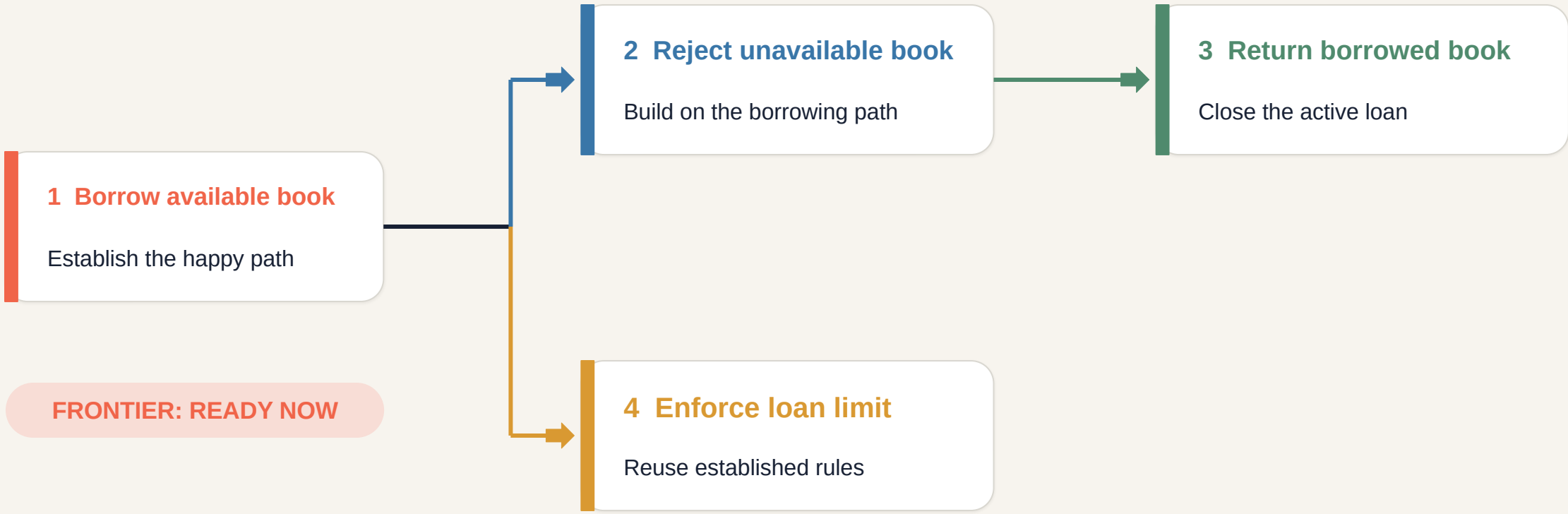
VERTICAL SLICE



Borrow one available book
from database to passing test

Small. Vertical. Testable.

The task graph



A graph reveals safe parallel work. A checklist hides dependencies.

Focused context for each task

Longer chats can obscure the details that matter.

One bounded task

Keep unrelated work out of the session.

Save the decisions

Preserve the spec, progress and open issues.

Start with a useful handoff

Load the ticket, relevant code and constraints.



Step 4: Pick an execution loop

1

MANUAL

You dispatch

Maximum visibility

learn

2

AGENT LOOP

Agent dispatches

Easy first AFK step

easy starting point

3

SCRIPTED LOOP

Code dispatches

Predictable orchestration

mature AFK

Pocock’s direction: deterministic orchestration wins after you tune it

Step 5: Give the code feedback

RED

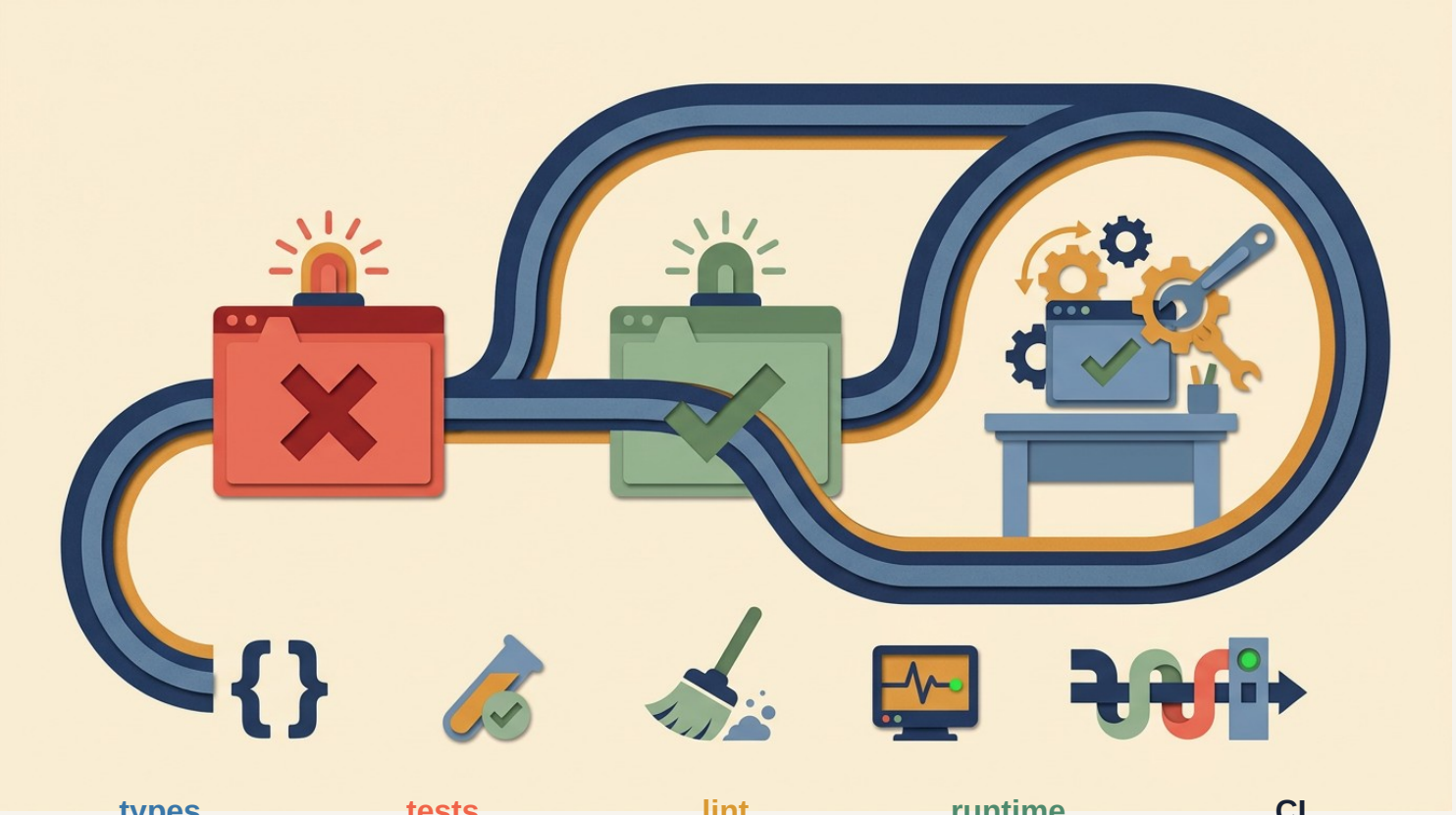
Start with a failing check

GREEN

Write the smallest code that passes

REFACTOR

Improve the design until it is a success



A failing check can stop the agent

Automated guardrails during implementation



/tdd

Failing test first

Express expected behavior. Watch the test fail, then implement the change.

/diagnosing-bugs

Reproduce the bug

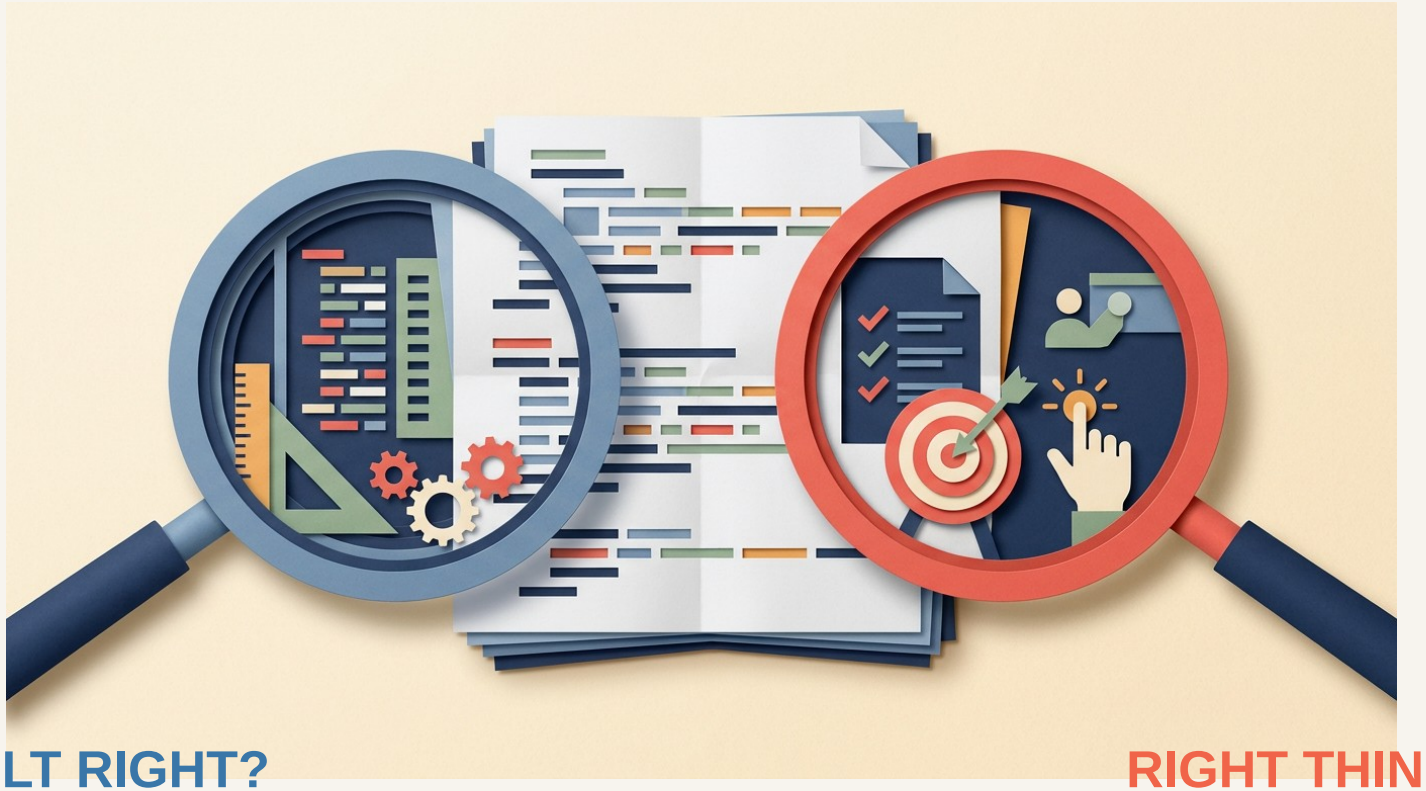
Capture the exact symptom. Test a cause before choosing a fix.

/implement

Verify before review

Run relevant tests and project checks. Report the results and any gaps.

Step 6: Review with two lenses



BUILT RIGHT?

Standards review

Conventions, design and maintainability

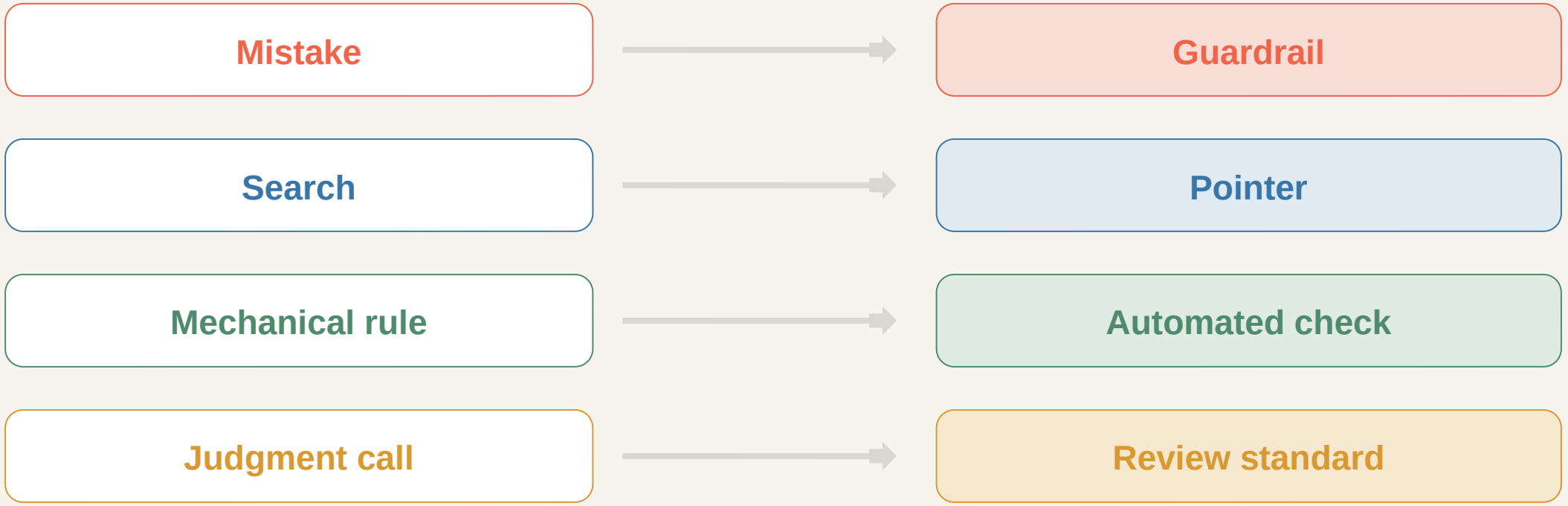
RIGHT THING?

Specification review

Behavior, scope and acceptance

Clean code can solve the wrong problem

Step 7: Make the next run better



Improve the environment, not only the prompt

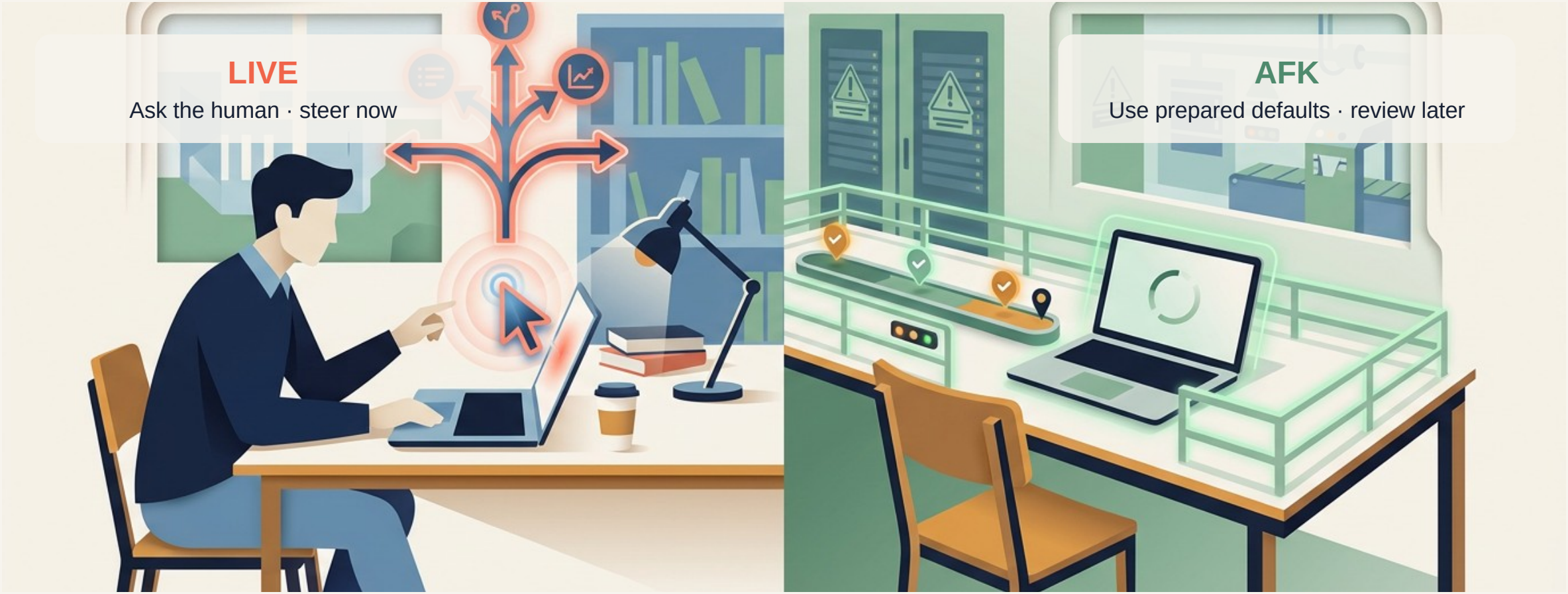
Copilot: the human remains the driver

1. **You choose the location**
2. **AI suggests the next edit**
3. **You accept or reject immediately**

Best for syntax, repetition and known designs



Live versus AFK



Start live. Earn the right to go AFK.

The AFK safety envelope



BEFORE

Clear spec + small tickets

DURING

Sandbox + checks + hard limit

AFTER

Reviewable branch

How much workflow do you want?

LIGHTER / FLEXIBLE

STRUCTURED / PRESCRIPTIVE



Bare harness

Plan mode, instructions, subagents

Pocock Skills

Choose small procedures

Spec Kit

Standard spec-driven sequence

GSD / BMAD

Full workflow systems

Less setup

More repeatability

No workflow is magic

Decisions can be lost

Conversation details weaken during synthesis

Agents can collide

Parallel branches still meet at merge time

Review can recurse

Agents may delegate more review agents

Plans can become waterfall

Too much wayfinding speculates far ahead

Workflows have bugs too

Your starter workflow

1. Explain
2. Question
3. Slice
4. Verify
5. Understand

Never submit code you cannot explain



Use the lightest workflow that protects the result and your learning

Thank you

Questions and discussion

Aleksandar Aytov

aleksandar.aytov@sap.com